CSS GRID

Grid Layout

The CSS Grid Layout Module offers a grid-based layout system, with rows and columns, making it easier to design web pages without having to use floats and positioning.

Browser Support

The grid properties are supported in all modern browsers.

57.0	16.0	52.0	10	44

Ejemplo :

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container1 {
  display:  grid;
  grid-template-columns: auto auto auto;
  background-color: #2196F3;
  padding: 10px;
}

.grid-item {
  background-color: rgba(255, 255, 255, 0.8);
  border: 1px solid rgba(0, 0, 0, 0.8);
  padding: 20px;
  font-size: 30px;
  text-align: center;
}
</style>
</head>
<body>

<h1>Grid Elements</h1>

<p>A Grid Layout must have a parent element with the <em>display</em> property set to <em>grid</em> or <em>inline-
grid</em>.</p>

<p>Direct child element(s) of the grid container automatically becomes grid items.</p>

<div class="grid-container1">
  <div class="grid-item">1</div>
  <div class="grid-item">2</div>
  <div class="grid-item">3</div>
  <div class="grid-item">4</div>
  <div class="grid-item">5</div>
  <div class="grid-item">6</div>
  <div class="grid-item">7</div>
  <div class="grid-item">8</div>
  <div class="grid-item">9</div>
</div>

</body>
```

El .grid-item contiene css normal (color de fondo, borde, etc).
El .grid-container1 contiene :
display : grid ; ==> Indica que usaremos css grid.
grid-template-columns : auto auto auto; ==> indica que usaremos 3 columnas de tamaño automático para introducir los elementos hijos de ese contenedor. Palabra “auto” separado por espacios en blanco.

Grid Elements

A Grid Layout must have a parent element with the *display* property set to *grid* or *inline-grid*.

Direct child element(s) of the grid container automatically becomes grid items.

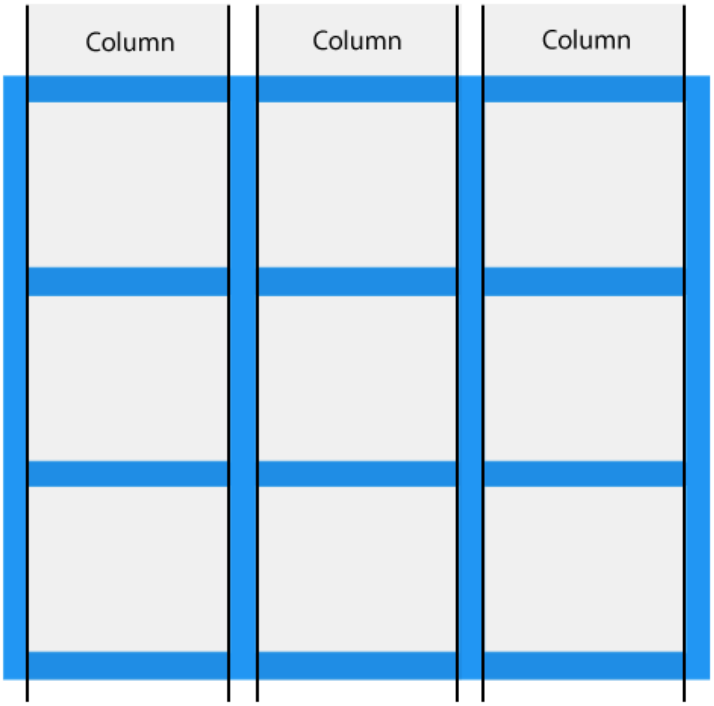
1	2	3
4	5	6
7	8	9

```
.grid-container {
  display: inline-grid;
```

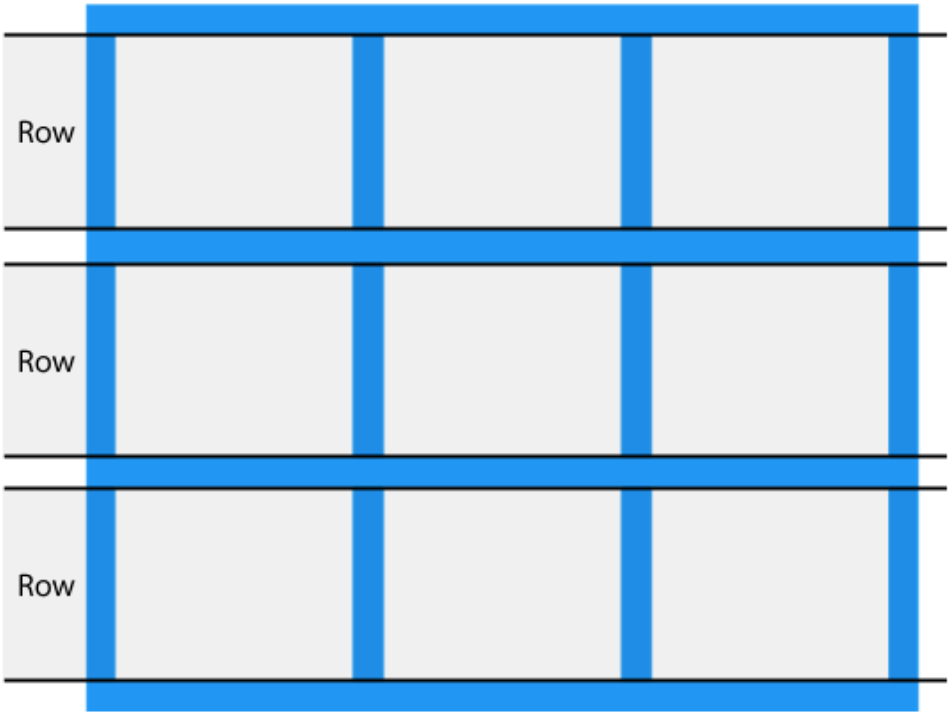
1	2	3
4	5	6
7	8	9

Las líneas verticales se denominan columnas :

The vertical lines of grid items are called *columns*.

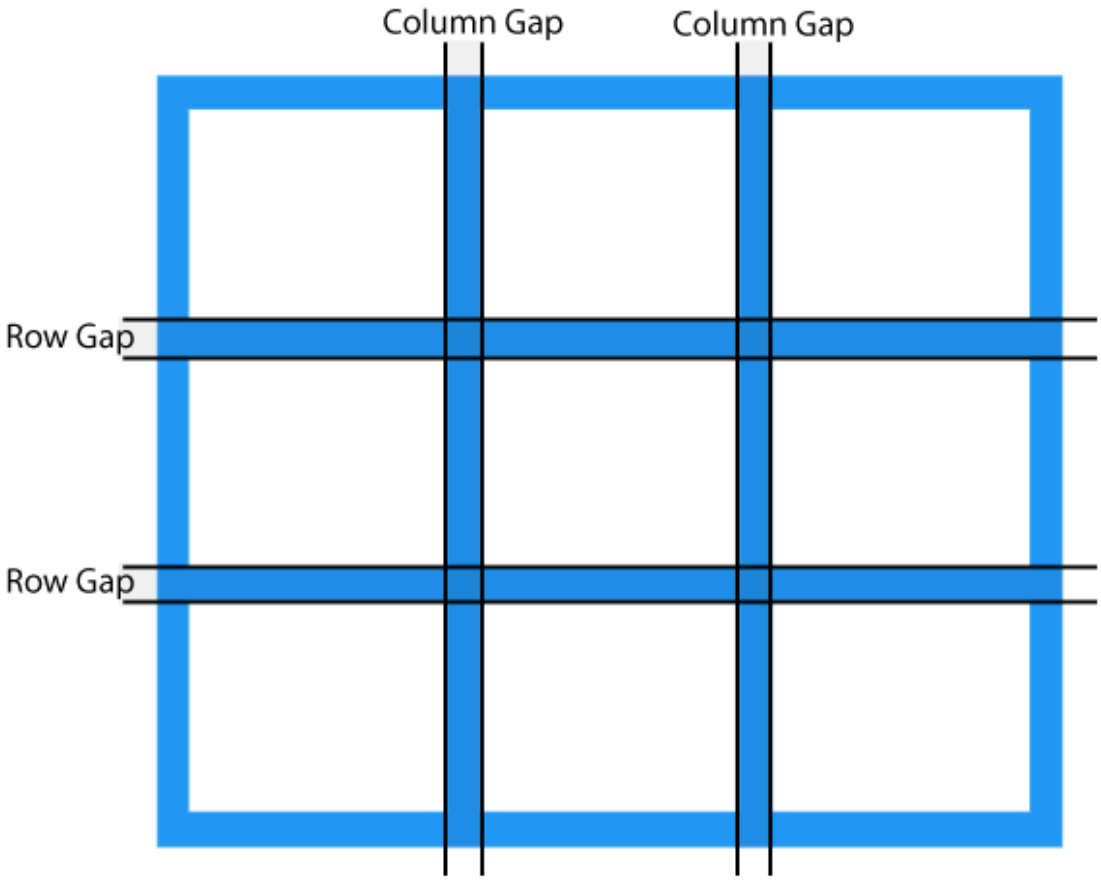


The horizontal lines of grid items are called *rows*.



Grid Gaps

The spaces between each column/row are called *gaps*.



You can adjust the gap size by using one of the following properties:

`grid-column-gap`
`grid-row-gap`
`grid-gap`

Ejemplo :

```html
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  grid-column-gap: 50px;
  grid-row-gap: 10px;
  grid-template-columns: auto auto auto;
  background-color: #2196F3;
  padding: 10px;
}

.grid-item {
  background-color: rgba(255, 255, 255, 0.8);
  border: 1px solid rgba(0, 0, 0, 0.8);
  padding: 20px;
  font-size: 30px;
  text-align: center;
}
</style>
</head>
<body>

<h1>The grid-column-gap Property:</h1>

<div class="grid-container">
  <div class="grid-item">1</div>
  <div class="grid-item">2</div>
  <div class="grid-item">3</div>
  <div class="grid-item">4</div>
  <div class="grid-item">5</div>
  <div class="grid-item">6</div>
  <div class="grid-item">7</div>
  <div class="grid-item">8</div>
  <div class="grid-item">9</div>
</div>
```

The grid-column-gap Property:

1	2	3
4	5	6
7	8	9

The *grid-gap* property is a shorthand property for the *grid-column-gap* and the *grid-row-gap* properties:

```
.grid-container {
  display: grid;
  grid-gap: 50px 100px;
}
```

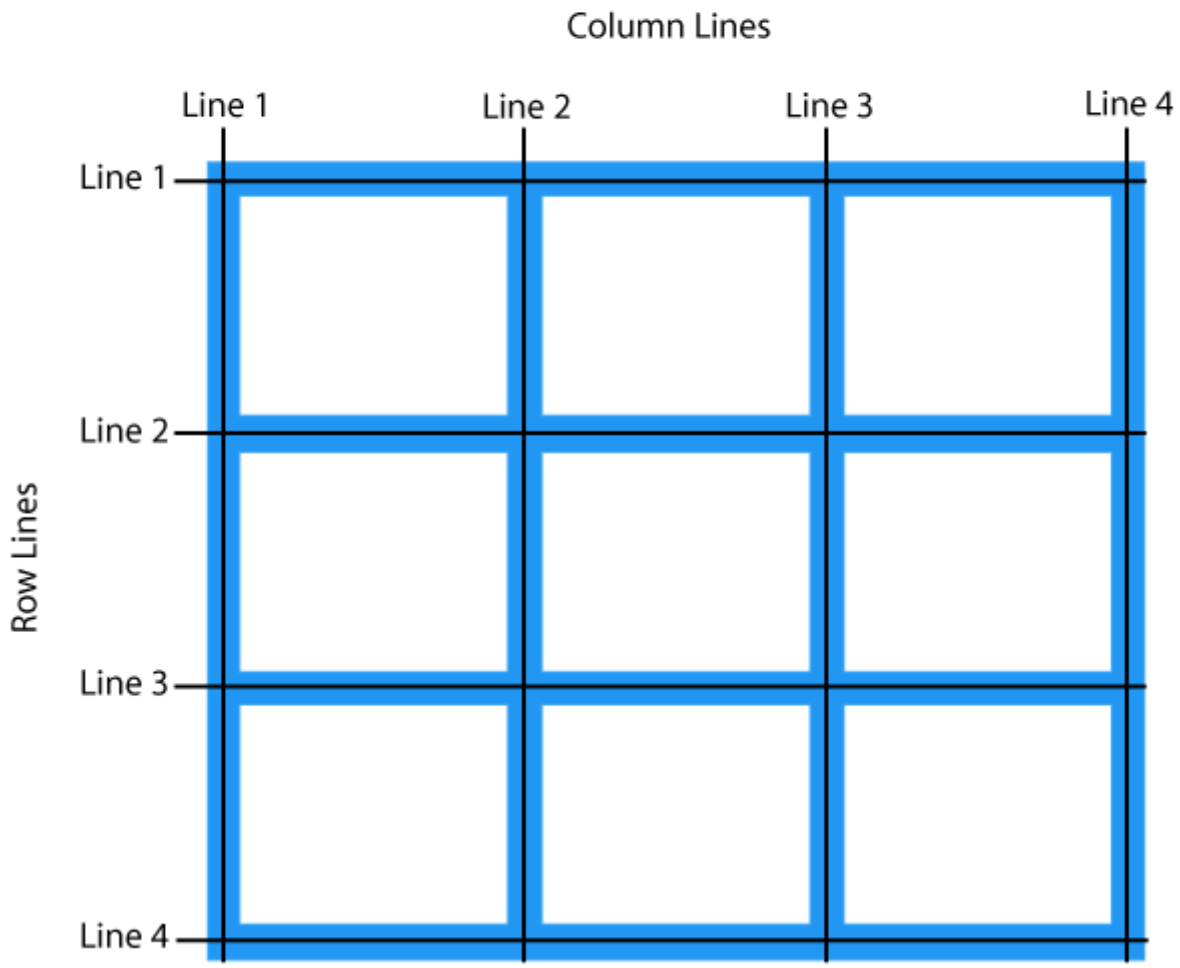
The *grid-gap* property can also be used to set both the row gap and the column gap in one value:

```
.grid-container {
  display: grid;
  grid-gap: 50px;
}
```

Grid Lines

The lines between columns are called *column lines*.

The lines between rows are called *row lines*.



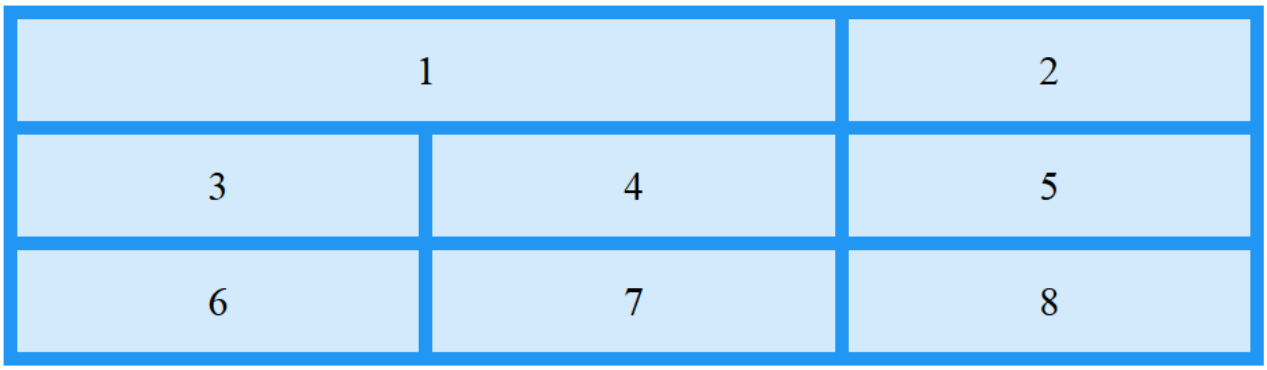
```html
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto;
  grid-gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  font-size: 30px;
}

.item1 {
  grid-row-start: 1;
  grid-row-end: 3;
}
</style>
</head>
<body>

<h1>Grid Lines</h1>

<div class="grid-container">
  <div class="item1">1</div>
  <div class="item2">2</div>
  <div class="item3">3</div>
  <div class="item4">4</div>
  <div class="item5">5</div>
  <div class="item6">6</div>
  <div class="item7">7</div>
  <div class="item8">8</div>
</div>
```

Grid Lines

<table>
<tr><td rowspan="2">1</td><td>2</td><td>3</td></tr>
<tr><td>4</td><td>5</td></tr>
<tr><td>6</td><td>7</td><td>8</td></tr>
</table>

```
.item1 {
  grid-column-start: 1;
  grid-column-end: 3;
}
```

Grid Lines



You can refer to line numbers when placing grid items.

The grid-template-columns Property

The `grid-template-columns` property defines the number of columns in your grid layout, and it can define the width of each column.

The value is a space-separated-list, where each value defines the length of the respective column.

If you want your grid layout to contain 4 columns, specify the width of the 4 columns, or "auto" if all columns should have the same width.

Example

Make a grid with 4 columns:

```
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto auto;
}
```

The `grid-template-columns` property can also be used to specify the size (width) of the columns.

Example

Set a size for the 4 columns:

```
.grid-container {
  display: grid;
  grid-template-columns: 80px 200px auto 40px;
}
```

```
.grid-container {
  display: grid;
  grid-template-columns: 80px 200px auto 30px;
  grid-gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>grid-template-columns</h1>

<p>Use the <em>grid-template-columns</em> property to specify the size of each column.</p>

<div class="grid-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
  <div>7</div>
  <div>8</div>
</div>
```

grid-template-columns

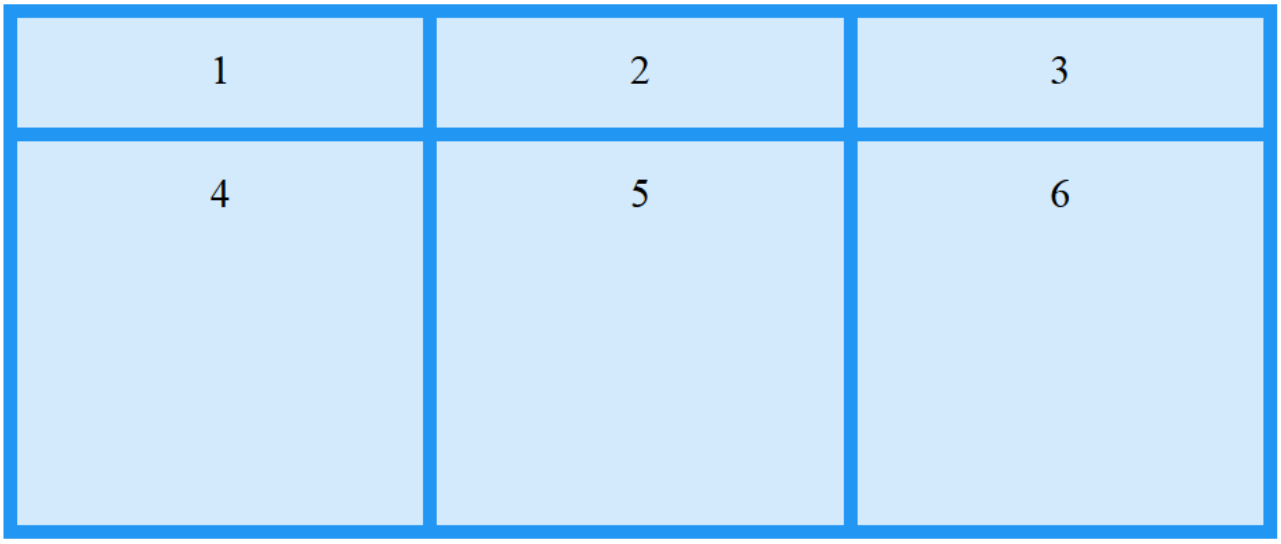
Use the *grid-template-columns* property to specify the size of each column.

1	2	3	4
5	6	7	8

```
.grid-container {
  display: grid;
  grid-template-rows: 80px 200px;
}
```

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  grid-template-columns: auto auto auto;
  grid-template-rows: 80px 280px;
  grid-gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>The grid-template-rows Property</h1>

<div class="grid-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
</div>
```

The grid-template-rows Property



The justify-content Property

The `justify-content` property is used to align the whole grid inside the container.

Note: The grid's total width has to be less than the container's width for the justify-content property to have any effect.

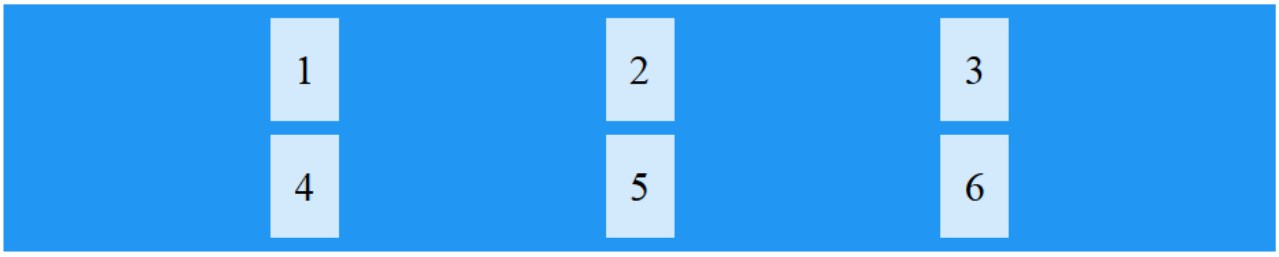
Example

```
.grid-container {
  display: grid;
  justify-content: space-evenly;
}
```

The justify-content Property

Use the *justify-content* property to align the grid inside the container.

The value "space-evenly" will give the columns equal amount of space between, and around them:

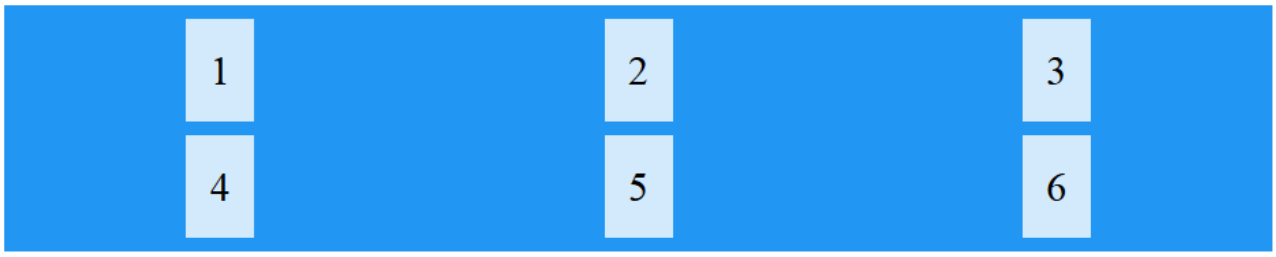


```
.grid-container {
  display: grid;
  justify-content: space-around;
}
```

The justify-content Property

Use the *justify-content* property to align the grid inside the container.

The value "space-around" will give the columns equal amount of space around them:

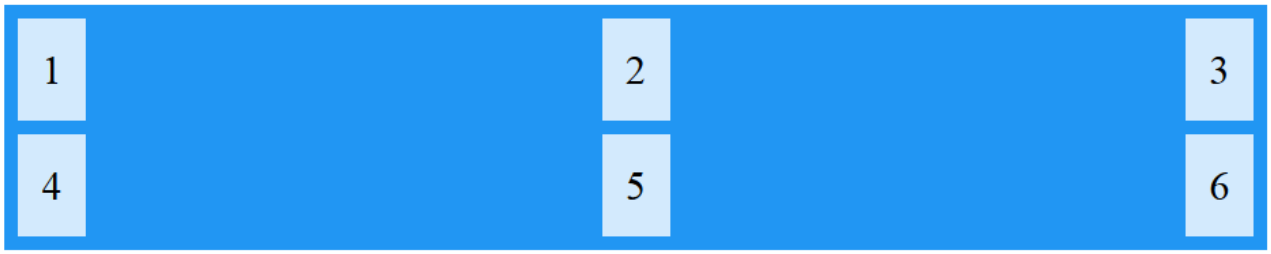



```
.grid-container {
  display: grid;
  justify-content: space-between;
}
```

The justify-content Property

Use the *justify-content* property to align the grid inside the container.

The value "space-between" will give the columns equal amount of space between them:

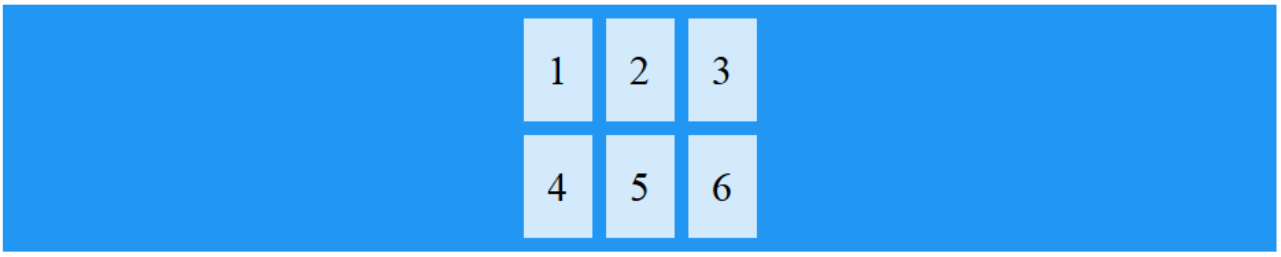


```
.grid-container {
  display: grid;
  justify-content: center;
}
```

The justify-content Property

Use the *justify-content* property to align the grid inside the container.

The value "center" will align the grid in the middle of the container:



start :

The justify-content Property

Use the *justify-content* property to align the grid inside the container.

The value "start" will align the grid at the beginning of the container:

end :

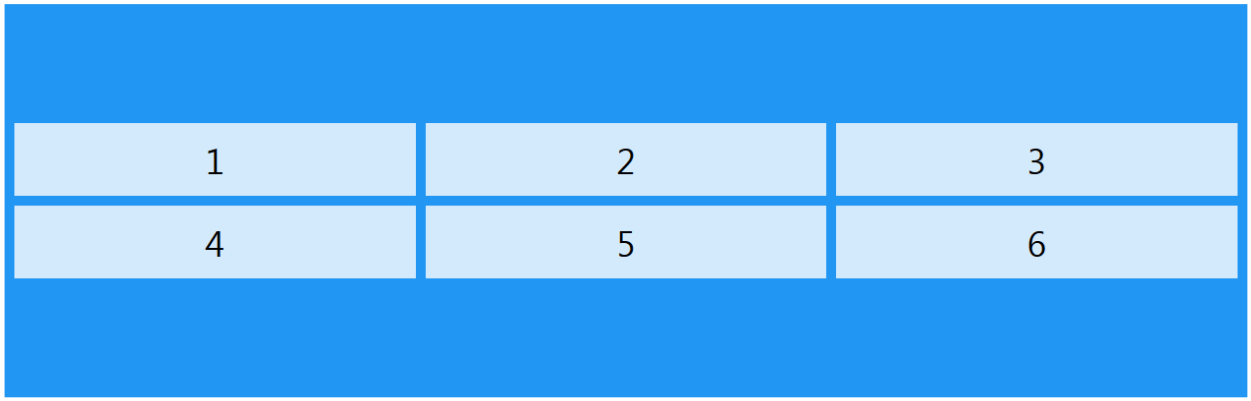
The justify-content Property

Use the *justify-content* property to align the grid inside the container.

The value "end" will align the grid at the end of the container:

The align-content Property

The `align-content` property is used to *vertically* align the whole grid inside the container.



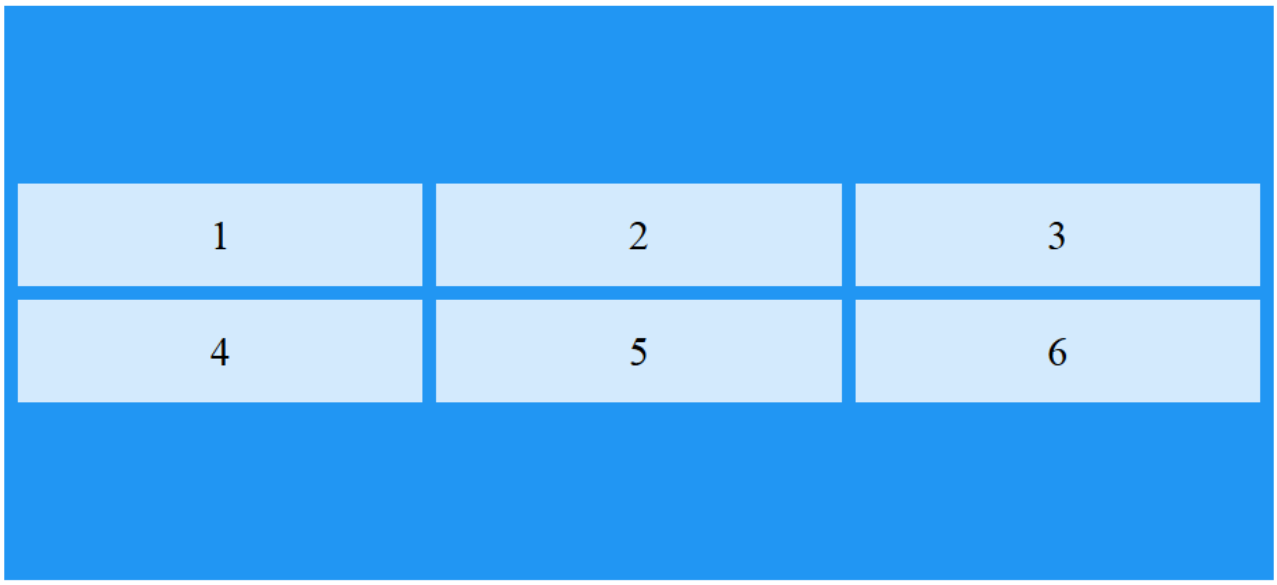
Note: The grid's total height has to be less than the container's height for the align-content property to have any effect.

```
<!DOCTYPE html>
<html>
<head>
<style>
.grid-container {
  display: grid;
  height: 400px;
  align-content: center;
  grid-template-columns: auto auto auto;
  grid-gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  font-size: 30px;
}
</style>
</head>
<body>

<h1>The align-content Property</h1>

<p>Use the <em>align-content</em> property to vertically align the grid inside the container.</p>

<p>The value "center" will align the rows in the middle of the container:</p>

<div class="grid-container">
  <div>1</div>
  <div>2</div>
  <div>3</div>
  <div>4</div>
  <div>5</div>
  <div>6</div>
</div>
```

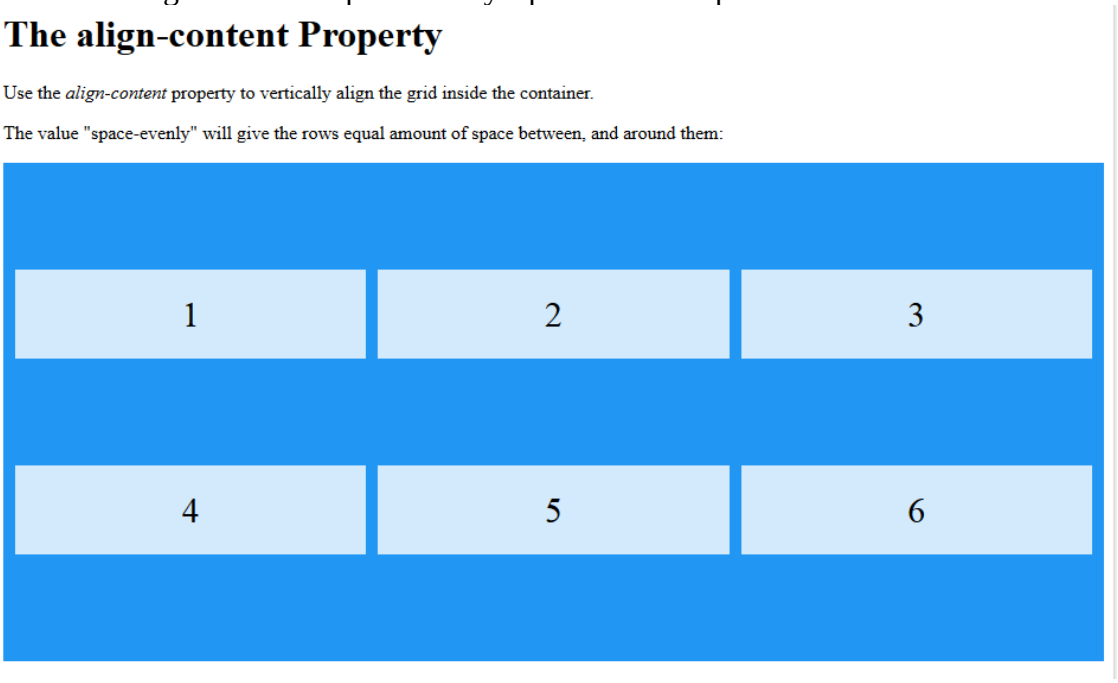
The align-content Property

Use the *align-content* property to vertically align the grid inside the container.

The value "center" will align the rows in the middle of the container:



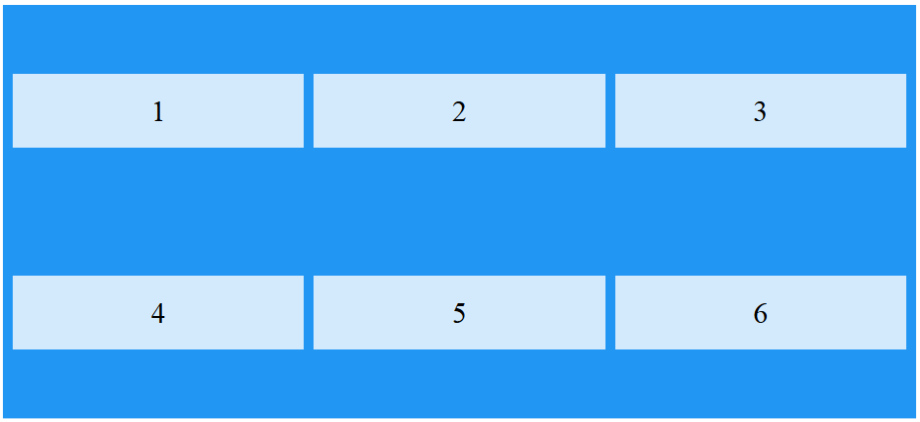
Otros valores de align-content : space-evenly space-around space-between start end



The align-content Property

Use the *align-content* property to vertically align the grid inside the container.

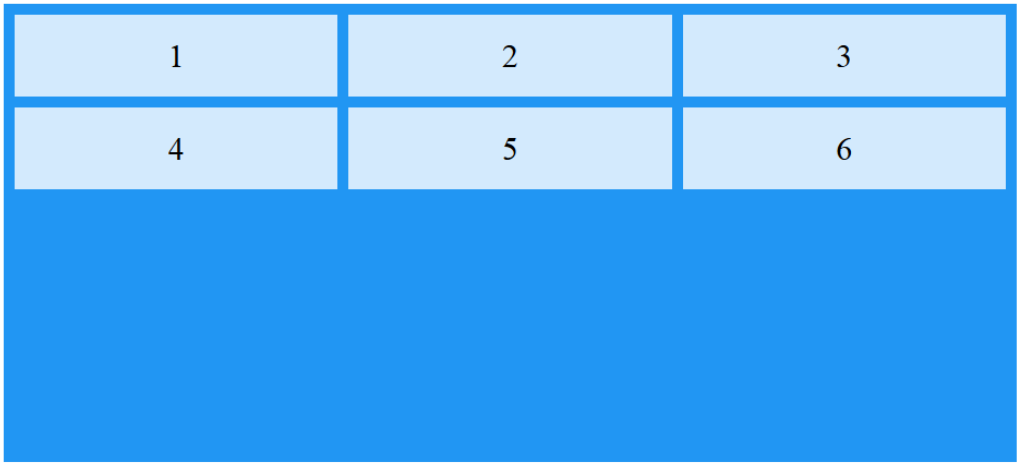
The value "space-around" will give the rows equal amount of space around them:



The align-content Property

Use the *align-content* property to vertically align the grid inside the container.

The value "start" will align the rows at the beginning of the container:



The grid-column Property:

The `grid-column` property defines on which column(s) to place an item.

You define where the item will start, and where the item will end.

Example

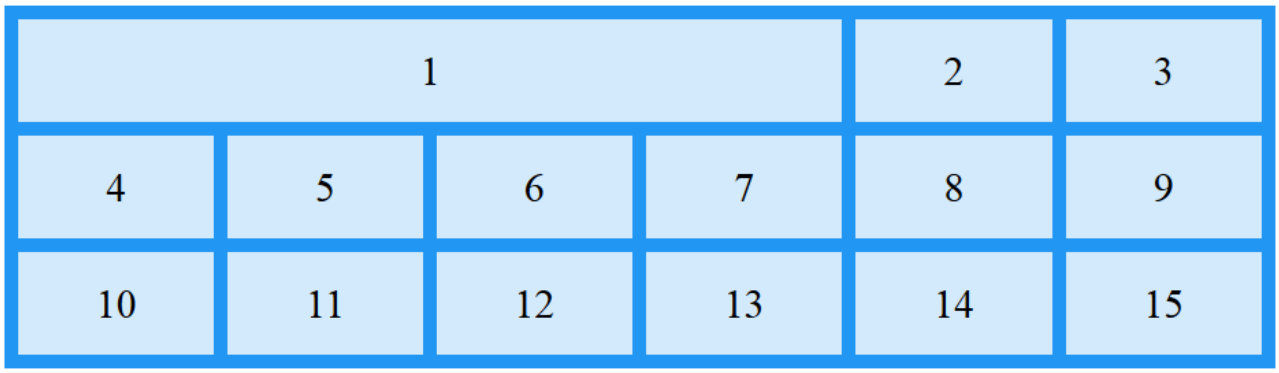
Make "item1" start on column 1 and end on column 5:

```
.item1 {
  grid-column: 1 / 5;
}
```

The grid-column Property

Use the *grid-column* property to specify where to place an item.

Item1 will start on column-line 1 and end on column-line 5:



El mismo resultado que el anterior. Empieza en la columna 1 y “ocupa” 4 columnas

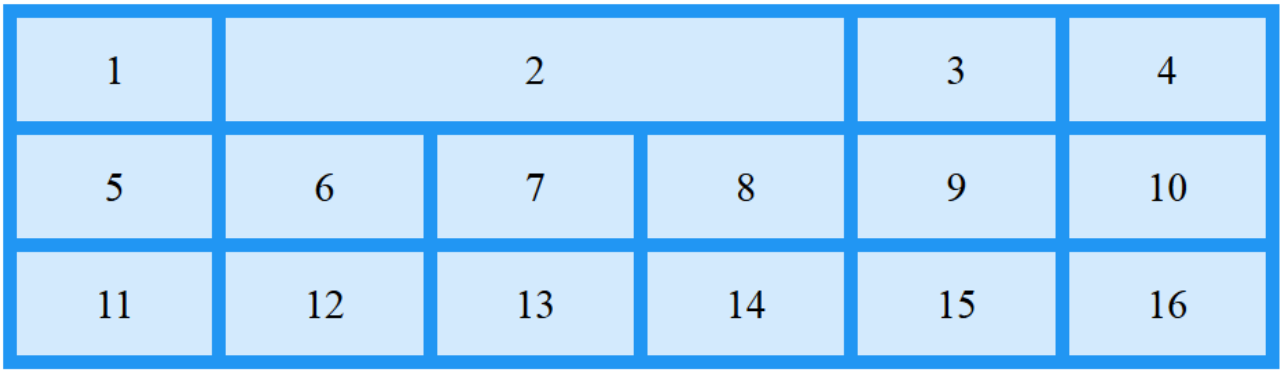
```
.item1 {
  grid-column: 1 / span 4;
}
```

```
.item2 {
  grid-column: 2 / span 3;
}
```

The grid-column Property

Use the *grid-column* property to specify where to place an item.

Item2 will start on column line 2 and span 3 columns:

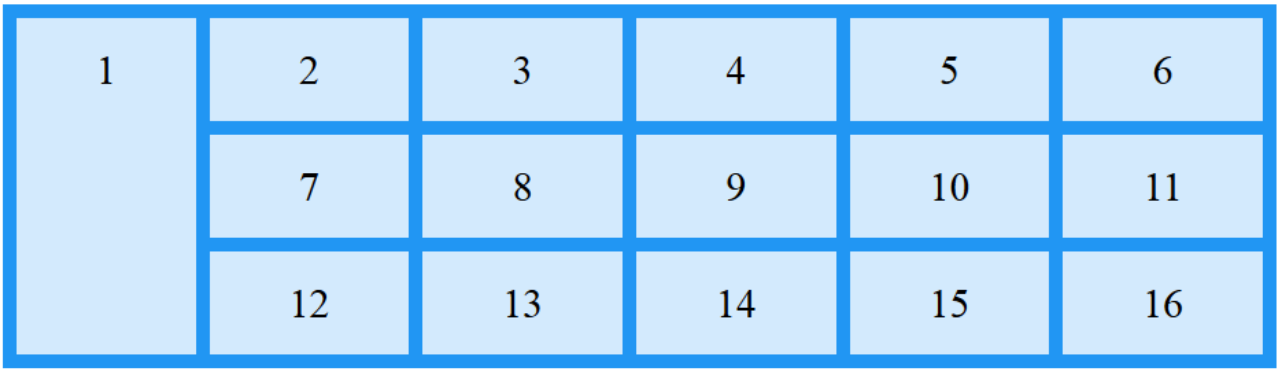



```
.item1 {
  grid-row: 1 / 4;
}
```

The grid-row Property

Use the *grid-row* property to specify where to place an item.

Item1 will start on row-line 1 and end on row-line 4:

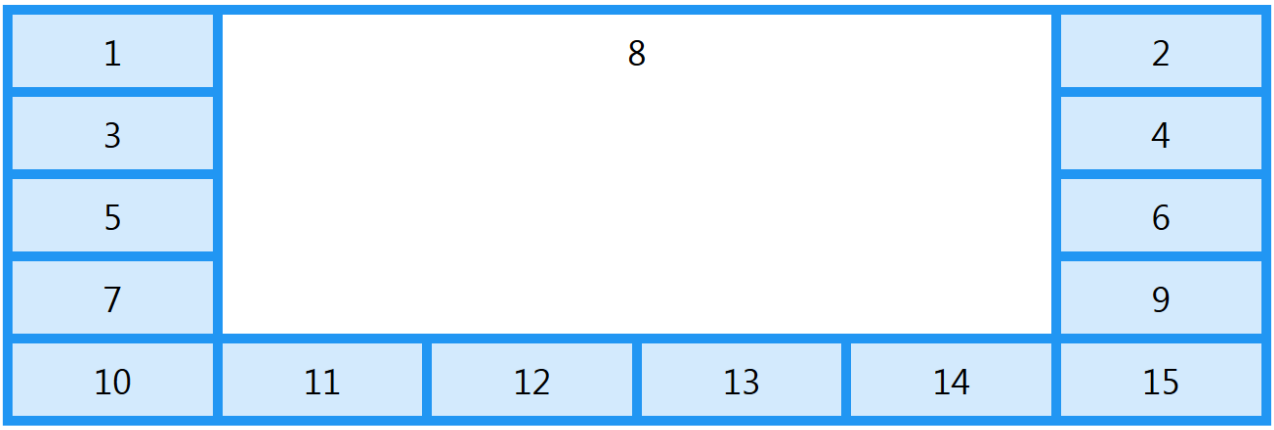


Mismo resultado que el anterior :

```
.item1 {
  grid-row: 1 / span 3;
}
```

The grid-area Property

The `grid-area` property can be used as a shorthand property for the *grid-row-start*, *grid-column-start*, *grid-row-end* and the *grid-column-end* properties.



Example

Make "item8" start on row-line 1 and column-line 2, and end on row-line 5 and column line 6:

```
.item8 {
  grid-area: 1 / 2 / 5 / 6;
}
```

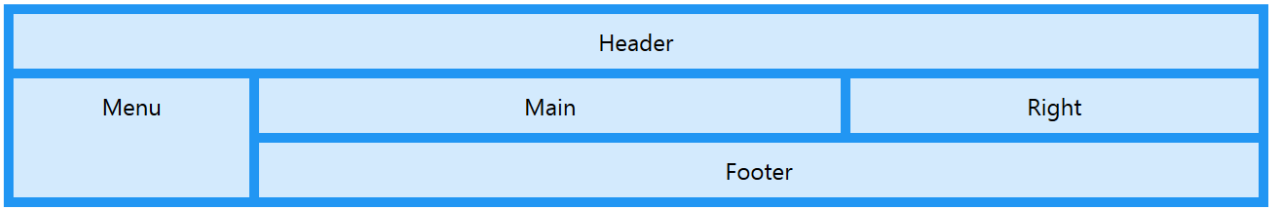
Mismo resultado con :

```
.item8 {
  grid-area: 1 / 2 / span 4 / span 4;
}
```

Area = Fila, columna (inicio), fila, columna (fin)

Naming Grid Items

The `grid-area` property can also be used to assign names to grid items.



Named grid items can be referred to by the `grid-template-areas` property of the grid container.

Example

Item1 gets the name "myArea" and spans all five columns in a five columns grid layout:

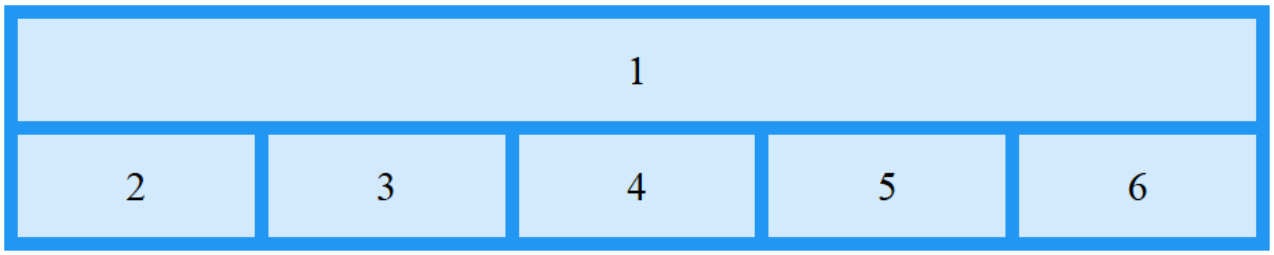
```
.item1 {
  grid-area: myArea;
}
.grid-container {
  grid-template-areas: 'myArea myArea myArea myArea myArea';
}
```

The grid-area Property

You can use the *grid-area* property to name grid items.

You can refer to the name when you set up the grid layout, by using the *grid-template-areas* property on the grid container.

Item1, is called "myArea" and will take up the place of all five columns:



Each row is defined by apostrophes (' ')

The columns in each row is defined inside the apostrophes, separated by a space.

> **Note:** A period sign represents a grid item with no name.

```
.item1 {
  grid-area: myArea;
}

.grid-container {
  display: grid;
  grid-template-areas: 'myArea myArea . . .';
  grid-gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  font-size: 30px;
```

The grid-area Property

You can use the *grid-area* property to name grid items.

You can refer to the name when you set up the grid layout, by using the *grid-template-areas* property on the grid container.

Item1, is called "myArea" and will take up the place of two columns (out of five):

1		2	3	4
5	6	7	8	9

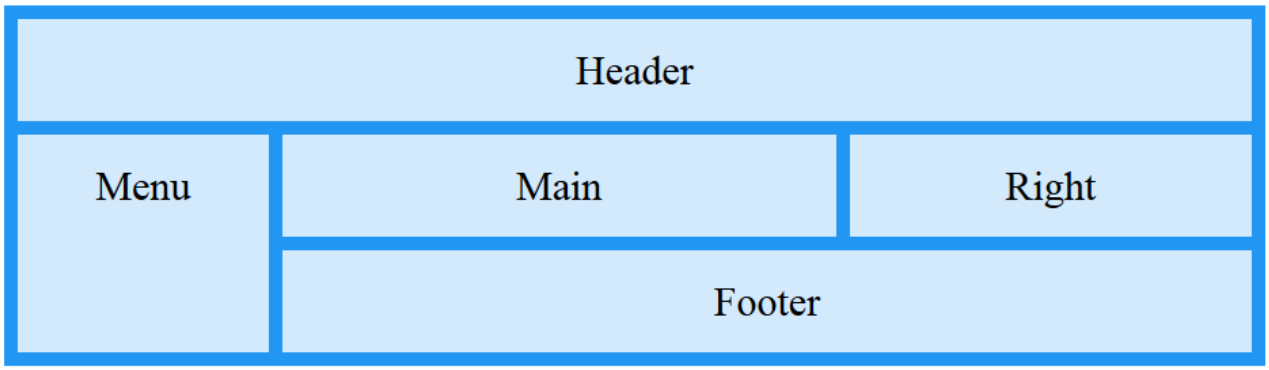
```html
<!DOCTYPE html>
<html>
<head>
<style>
.item1 { grid-area: header; }
.item2 { grid-area: menu; }
.item3 { grid-area: main; }
.item4 { grid-area: right; }
.item5 { grid-area: footer; }

.grid-container {
  display: grid;
  grid-template-areas:
    'header header header header header header'
    'menu main main main right right'
    'menu footer footer footer footer footer';
  grid-gap: 10px;
  background-color: #2196F3;
  padding: 10px;
}

.grid-container > div {
  background-color: rgba(255, 255, 255, 0.8);
  text-align: center;
  padding: 20px 0;
  font-size: 30px;
}
</style>
</head>
<body>
```

The grid-area Property

You can use the *grid-area* property to name grid items.

You can refer to the name when you set up the grid layout, by using the *grid-template-areas* property on the grid container.

This grid layout contains six columns and three rows:



Otras consideraciones

grid-auto-columns sets a size for the columns in a grid container.

Esta propiedad afecta solo a las columnas sin tamaño especificado (implícitas).

Ejemplo : grid-auto-columns: 50px;

Otros valores :

max-content Sets the size of each column depending on the largest item in the column
min-content Sets the size of each column depending on the smallest item in the column

grid-auto-rows sets a size for the rows in a grid container.
Esta propiedad afecta solo a las filas sin tamaño específico.

Ejemplos :

```
grid-auto-rows: 150px;
grid-auto-rows: min-content;
grid-auto-rows: max-content;
```

grid-auto-* ==> Se refiere a aquellas columnas o filas donde no se especificó un tamaño de forma explícita. En el siguiente ejemplo hay definidas dos filas de forma explícita de 100px. Para el resto de ellas el tamaño es de 30px.

```
.grid {
  display: grid;
  grid-gap: 10px; /* add spacing for better visibility */
  grid-auto-rows: 30px;
  grid-template-rows: 100px 100px;
}
```

En el siguiente ejemplo estoy usando minmax() en el valor de la propiedad grid-auto-rows. Las filas creadas **automáticamente** tendrán como mínimo de 100 píxeles de alto y un máximo de auto. El uso de auto significa que el tamaño mirará el tamaño del contenido y se estirará para dar espacio al elemento más alto en una celda en esta fila.

```
.wrapper {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-auto-rows: minmax(100px, auto);
}
```

Unidad de medida : fr (fracción)

```
.wrapper {
  display: grid;
  grid-template-columns: 2fr 1fr 1fr;  // El espacio se divide en 4 partes
                                       // Cada columna lleva la parte indicada
```

En este ejemplo final mezclamos las vías de tamaño absoluto con unidades de fracción(fr). La primera vía tiene 500 píxeles, por lo que este ancho fijo se sustrae del espacio disponible. El espacio restante se divide en tres y se asigna en proporción a las dos columnas flexibles.

```
.wrapper {
  display: grid;
  grid-template-columns: 500px 1fr 2fr;
}
```

repeat

```
grid-template-columns: repeat(3, 1fr);
```

```
equivale a
      grid-template-columns: 1fr 1fr 1fr;
```

En el siguiente ejemplo he creado una cuadrícula con una vía inicial de 20 píxeles luego una sección repetitiva de 6 vías de `1fr` y luego una vía final de 20 píxeles.

```
grid-template-columns: 20px repeat(6, 1fr) 20px;
```

La notación de repetición toma una lista de vías específicas, por lo tanto, puede utilizarla para crear un patrón de iteración de vías. En el próximo ejemplo, mi cuadrícula consistirá de 10 vías, una vía `1fr` seguida por una vía `2fr`, repetida cinco veces.

```
grid-template-columns: repeat(5, 1fr 2fr);
```

min-max(min, max)

La función minmax() en CSS define un rango de tamaño mayor o igual que min y menor o igual que max

Ejemplos :

```
minmax(200px, 1fr)
minmax(400px, 50%)
minmax(30%, 300px)
minmax(auto, 300px)
```

Define 3 columnas utilizando minmax :

```
grid-template-columns: minmax(100px, 300px) minmax(200px, 1fr) 150px;
```

Ejemplo página responsive con css-grid :

https://developer.mozilla.org/es/docs/Web/CSS/CSS_Grid_Layout/Conceptos_B%C3%A1sicos_del_Posicionamiento_con_Rejillas

En la carpeta ResponsiveCssGrid encontrarás el siguiente ejemplo :

```html
<!DOCTYPE html>
<html lang="es">
    <head>
        <link href="responsivegrid.css" rel="stylesheet"/>
        <title>Grid responsive</title>
        <meta charset="utf-8">
    </head>
    <body>
     <div class="wrapper">
            <header class="main-head caja">The header</header>
            <nav class="main-nav caja">
                <ul>
                    <li><a href="">Nav 1</a></li>
                    <li><a href="">Nav 2</a></li>
                    <li><a href="">Nav 3</a></li>
                </ul>
            </nav>
            <article class="content caja">
                <h1>Área principal del artículo</h1>
                <p>En este diseño, las áreas se muestran en el orden en que están escritas en las pan
            </article>
            <aside class="side caja">Sidebar</aside>
            <div class="ad caja">Advertising</div>
            <footer class="main-footer caja">The footer</footer>
    </div>
    </body>
</html>
```

```
.caja {
  border: 2px solid #f08c00;
  background-color: #ffec99;
  border-radius: 5px;
  padding: 10px;
}

.main-head {
  grid-area: header;
}
.content {
  grid-area: content;
}
.main-nav {
  grid-area: nav;
}
.side {
  grid-area: sidebar;
}
.ad {
  grid-area: ad;
}
.main-footer {
  grid-area: footer;
}
.wrapper {
  display: grid;
  grid-gap: 20px;
  grid-template-areas:
    "header"
    "nav"
    "content"
    "sidebar"
    "ad"
    "footer";
}
```

Una plantilla (hasta ahora) en la que indicamos que la cabecera, barra navegación, contenido, etc ocupe una fila (observa .wrapper)

```
@media (min-width: 500px) {
  .wrapper {
    grid-template-columns: 1fr 3fr;
    grid-template-areas:
      "header  header"
      "nav     nav"
      "sidebar content"
      "ad      footer";
  }
  nav ul {
    display: flex;   /* Veremos flex  */
    justify-content: space-between;
  }
}

@media (min-width: 700px) {
  .wrapper {
    grid-template-columns: 1fr 4fr 1fr;
    grid-template-areas:
      "header header  header"
      "nav    content sidebar"
      "nav    content ad"
      "footer footer  footer"
   }
   nav ul {
     flex-direction: column;
   }
}
```

La clave está en el @media que indica que cuando la pantalla tenga un mínimo de 500px aplique unos parámetros a la clase .wrapper, cuando tenga un mínimo de 700px otros y en caso contrario (pantalla inferior a 500px) deja los valores indicados (que son los de una columna).

Podemos nombrar las líneas de las columnas o filas. No lo veremos por ahora. Para mas información :

https://developer.mozilla.org/es/docs/Web/CSS/CSS_Grid_Layout/Layout_using_Named_Grid_Lines

Ver :

https://gridbyexample.com/

Ejemplos avanzados :

https://webdesign.tutsplus.com/es/tutorials/create-a-broken-grid-layout-using-css-grid--cms-30870